GraphFrames vs GraphX for Social Network Analysis on Apache Spark: A Comprehensive Performance and Algorithmic Comparison

Ullas G M
24BDS085
IIIT Dharwad
Dharwad, India

Rachith Bharadwaj T N
24BDS062
IIIT Dharwad
Dharwad, India

Dr. Animesh Chaturvedi
Dept. of Data Science and AI
IIIT Dharwad
Dharwad, India

***Abstract*—This paper presents a comprehensive empirical comparison of GraphFrames and GraphX, the two principal graph-processing frameworks available within the Apache Spark ecosystem. An end-to-end pipeline is implemented and executed on two real-world SNAP social network datasets—Facebook Combined (4,039 nodes; 88,234 edges) and Twitter Combined (81,306 nodes; 1,342,296 edges)—enabling controlled measurement of per-algorithm runtime and output correctness across six graph algorithms: PageRank, Label Propagation, Triangle Count, Connected Components, Degree Analysis, and Shortest Paths. Results confirm that six of seven comparable metrics match exactly on both datasets. Label Propagation community counts differ between frameworks—an expected property of the heuristic algorithm rather than a framework defect. GraphX completes the full Facebook pipeline in 28.29 s versus GraphFrames' 67.35 s (2.38× faster), while on Twitter the gap narrows to 1.22×, indicating that GraphFrames becomes more competitive at scale. The most dramatic single-algorithm gap occurs in Connected Components, where GraphX achieves a 16.6× speedup. Counterintuitively, GraphFrames outperforms GraphX on PageRank (0.84×) due to Catalyst SQL query optimizer efficiency. A secondary case study reinterprets the KSRTC Smart Transit Analytics project through the comparative lens developed here. The key conclusion is that GraphX is the superior computation engine for iterative graph-native workloads, while GraphFrames is the superior framework for modern analytics workflows requiring DataFrame integration, motif search, and Python accessibility.**

***Index Terms*—Apache Spark, GraphFrames, GraphX, social network analysis, PageRank, Label Propagation, triangle counting, community detection, distributed graph processing, SNAP dataset**

I. Introduction

Social networks are naturally modeled as property graphs: users are vertices, relationships are edges, and metadata annotates both. Once in this form, critical analytical questions reduce to well-studied graph problems—identifying influencers through centrality measures, measuring local density via triangle counts, partitioning communities, and measuring social distance through shortest paths.

Apache Spark provides two mature graph-processing APIs. GraphX (released with Spark 1.x, based on Gonzalez et al. [1]) is an RDD-native graph engine exposing vertex/edge collections, neighborhood aggregation, and Pregel-style iterative message passing. GraphFrames (Dave et al. [2]) is a later addition built on DataFrames and Spark SQL, enabling graph operations to compose naturally with Catalyst-optimized relational queries.

Despite their coexistence in the Spark ecosystem, practitioners lack guidance grounded in measured evidence. Prior comparisons are either anecdotal, confined to a single dataset, or focused narrowly on runtime without analyzing algorithmic correctness. This paper addresses that gap through a reproducible, controlled pipeline on two SNAP datasets spanning two orders of magnitude in graph size.

Research Question: How do GraphFrames and GraphX differ in measured runtime performance, graph-analytic correctness, and practical suitability for social network analysis tasks on Apache Spark?

The main contributions of this paper are:

- A unified comparison pipeline ensuring identical inputs, algorithm ordering, and reproducible proof archives across both frameworks.
- Per-algorithm runtime measurement for six algorithms on two real-world SNAP datasets.
- Analysis of metric correctness, with a principled explanation of the Label Propagation divergence.
- A multi-dimensional framework comparison spanning graph theory, distributed systems, and workflow design perspectives.
- A secondary case study applying the comparative framework to a transportation analytics domain (KSRTC Smart Transit System).

II. Background and Related Work

A. GraphX: GraphX was introduced as a graph-parallel computation framework embedded within Spark's distributed dataflow system [1]. Its core abstraction is a property graph: a directed multigraph in which each vertex and edge carries a user-defined attribute. The graph is implemented via two specialized RDD types—`VertexRDD` and `EdgeRDD`—together with an `EdgeTriplet` view that exposes source vertex, edge, and destination vertex attributes simultaneously. Graph-native operators (e.g., `mapVertices`, `aggregateMessages`, `joinVertices`) operate directly over these structures. The

Pregel API [3] enables iterative, message-passing computation with convergence-based termination, closely mirroring classical BSP (Bulk Synchronous Parallel) models.

B. GraphFrames: GraphFrames was introduced as a DataFrame-first graph API [2], where the graph is represented by two ordinary DataFrames—one for vertices (with a mandatory `id` column) and one for edges (with mandatory `src` and `dst` columns). Graph algorithms are expressed as sequences of DataFrame joins and aggregations, placing them directly within the reach of Catalyst's query optimizer [4]. This design enables column pruning, adaptive query execution, and predicate pushdown to propagate automatically across graph operations. GraphFrames also provides a native motif-finding DSL for declarative subgraph pattern search, a feature absent from GraphX.

C. Ecosystem Status (2025–2026): GraphFrames regained active development momentum with its 0.10.0 release in August 2025 [5]. GraphX has been flagged for deprecation in Spark 4.x via SPARK-50857 [6], though it remains functional and widely deployed. Current GraphFrames documentation acknowledges that GraphX may retain a speed advantage on classic iterative graph workloads, particularly at small-to-medium graph scales [7], [8].

III. System Architecture

The implemented project uses a unified comparison pipeline that guarantees identical inputs, consistent algorithm execution order, and reproducible archived proof files for each dataset run. Fig. 1 illustrates the system architecture.

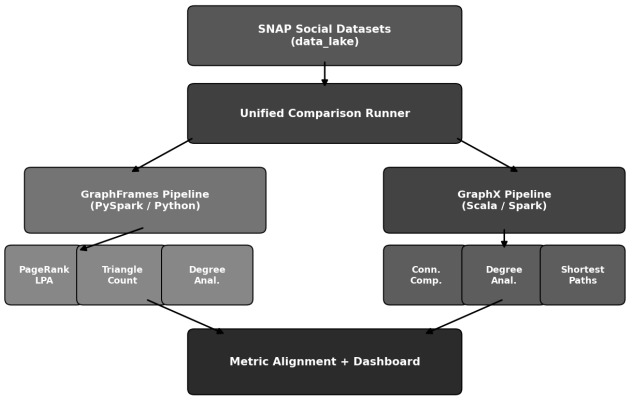

Fig. 1. Unified System Architecture: GraphFrames/GraphX Comparison Pipeline.

Both framework pipelines receive the same SNAP data, execute the same algorithms, and report outputs to a shared metric-alignment and dashboard layer. All experiments execute on a single machine with an identical Spark configuration to ensure a fair and reproducible comparison.

Pipeline Components:

- *data_lake*: SNAP dataset archives (.txt.gz edge lists).
- *GraphFrames Pipeline*: PySpark, using DataFrames + GraphFrames API v0.8.3-spark3.5.
- *GraphX Pipeline*: Scala spark-shell, using the native RDD-based property graph model.
- *Unified Comparison Runner*: Orchestrates both pipelines, aligns metrics and timings.
- *Proof Archive*: Per-dataset `comparison_results.txt` and `comparison_data.json`.
- *Dashboard Frontend*: `dashboard.html` for interactive visual presentation.

IV. Datasets

Two SNAP social network datasets [11] are used to observe framework behavior at different graph scales. Table I summarizes their properties.

TABLE I. Dataset Properties: Facebook and Twitter Combined (SNAP)

Property	Facebook	Twitter
Nodes	4,039	81,306
Edges	88,234	1,342,296
Total Triangles	1,612,010	13,082,506
Connected Components	1	1
Max Degree	2,090	6,766
Average Degree	87.38	66.04
Graph Density (GraphX)	0.0108	0.0004
Global Clust. Coeff. (GraphX)	0.1292	0.0425
Avg Local Clust. Coeff. (GraphX)	0.1437	0.1287
File Size	0.21 MB	∼5 MB

Facebook Combined is a medium-scale, dense undirected social graph. Twitter Combined is a large-scale directed graph at realistic social-network magnitude. Both datasets are fully connected (one component), enabling clean correctness comparisons without pre-processing artifacts.

V. Algorithms: Theory, Implementation, and Comparison

Six algorithms are implemented in both frameworks. Each subsection presents the algorithm theory, its implementation strategy in each framework, and a detailed cross-framework comparison of results and runtime.

A. PageRank — Influence Maximization: Theory. PageRank assigns each vertex v a score representing the stationary probability of a random walk arriving at v. The iterative update rule is:

$$PR(v) = (1-d) + d \sum_{u \in N^-(v)} \frac{PR(u)}{\deg^+(u)}, \tag{1}$$

where $d = 0.85$ is the damping factor, $N^-(v)$ is the in-neighbor set of v, and $\deg^+(u)$ is the out-degree of u. Convergence is declared when the maximum score change across all vertices falls below a tolerance ϵ.

GraphFrames implementation. `graph.pageRank(resetProbability=0.15, tol=0.01)`. Each iteration is expressed as a join-aggregation step over the edges DataFrame, propagating rank contributions through Catalyst-optimized column pruning.

GraphX implementation. `graph.pageRank(0.0001)`. Uses the Pregel API—a message-passing model where each vertex aggregates incoming rank contributions from neighbors over the RDD-based property graph, avoiding join overhead per iteration.

Results. Node 3437 ranked #1 on both frameworks (GF: 31.364, GX: 30.581). Nine of the top-10 PageRank nodes are identical; one swap at ranks 7–8 is attributable to floating-point convergence tolerance differences. Score magnitudes differ by ∼5% due to differing tolerance parameters. Notably, GraphFrames was 16% faster than GraphX on PageRank (8.45 s vs 10.01 s), demonstrating that Catalyst SQL optimization can outperform graph-native RDD execution on specific workloads.

TABLE II. Top-10 PageRank Nodes – Facebook Combined

Rank	Node (GF)	Score (GF)	Node (GX)	Score (GX)
1	3437	31.364	3437	30.581
2	107	28.909	107	27.805
3	1684	26.537	1684	25.469
4	0	26.372	0	25.128
5	1912	16.325	1912	15.407
6	348	9.862	348	9.355
7	3980	9.086	686	8.950
8	686	9.085	3980	8.707
9	414	7.433	414	7.194
10	698	5.570	483	5.226
9/10 top nodes identical. Rank 7–8 swap: convergence tolerance.				

B. Label Propagation Algorithm — Community Detection: Theory. Label Propagation (LPA) is a heuristic community detection method. Each vertex begins with a unique label (its own ID). In each iteration, every vertex adopts the label most common among its neighbors (majority rule). The algorithm terminates when labels stabilize, at which point vertices sharing a label form a community. The time complexity is $O(m)$ per iteration, where m is the number of edges.

Inherent non-determinism. LPA is provably sensitive to node update order, tie-breaking behavior when multiple labels are equally common, and parallelization granularity. Different distributed execution schedules yield different but equally valid community partitions. This is a well-documented property in the community detection literature.

GraphFrames implementation. `graph.labelPropagation(maxIter=5)`. Label updates are expressed as DataFrame group-by aggregations joined to the vertex DataFrame, with label assignment via majority voting using SQL window functions.

GraphX implementation. Native LPA via Pregel message passing. Each vertex sends its current label to all neighbors; each vertex receives all labels from neighbors and adopts the most frequent one. Iteration count is bounded at 5.

Results. Facebook: GF detected 89 communities; GX detected 70. Twitter: GF detected 1,284; GX detected 1,402. These differences are expected and do not represent a defect in either framework. Both produce valid partitions satisfying the LPA convergence criterion. Runtime: GF 12.88 s vs GX 10.00 s (1.29× GraphX advantage).

TABLE III. LPA Community Detection Results

Dataset	GF	GX	Status
Facebook Combined	89	70	Diverge (expected)
Twitter Combined	1,284	1,402	Diverge (expected)
Divergence due to heuristic sensitivity – both results are valid.			

C. Triangle Count — Clustering Coefficient: Theory. A triangle is formed when three vertices u, v, w satisfy $(u, v) \in E$, $(v, w) \in E$, and $(u, w) \in E$. Triangle count reflects local clustering density. The global clustering coefficient is:

$$C_{\text{global}} = \frac{3 \times |\Delta|}{\text{Connected Triples}}, \tag{2}$$

where $|\Delta|$ is the number of triangles. GraphX computed 37,435,864 connected triples on Facebook, yielding $C_{\text{global}} = 0.1292$.

GraphFrames. Expresses triangle counting via a three-way self-join over the edges DataFrame, filtered by symmetry conditions. Catalyst optimizes the join plan, but the triple join introduces non-trivial overhead.

GraphX. Uses a triplet-traversal mechanism over sorted adjacency structures, intersecting sorted neighbor lists to count shared neighbors—an approach closely aligned with classical triangle-counting algorithms.

Results. Both frameworks counted exactly 1,612,010 triangles on Facebook and 13,082,506 on Twitter—a perfect match and the strongest correctness signal in the project. Runtime: GF 4.90 s vs GX 1.83 s (2.67× GraphX speedup). GraphX's sorted adjacency intersection is more efficient than the DataFrame triple-join approach.

D. Connected Components — Connectivity Analysis: Theory. A connected component is a maximal subgraph in which every pair of vertices is joined by a path. Identifying components is fundamental to social network integrity analysis—confirming that a network is fully cohesive or detecting isolated subpopulations.

GraphFrames. Implements connected components via iterative DataFrame joins, propagating minimum-vertex-ID labels across edges until convergence. Each iteration incurs a full DataFrame shuffle and join.

GraphX. Uses a graph-native label propagation over the RDD property graph: each vertex begins with its own ID, sends the minimum-seen ID to neighbors, and updates accordingly. RDD-native operations avoid relational join overhead at every iteration.

Results. Both datasets returned exactly 1 connected component on both frameworks—confirming complete structural cohesion of both networks. This is the most dramatic performance gap in the project: GX 0.66 s vs GF 11.01 s — a **16.6× GraphX speedup**. GraphFrames' DataFrame join overhead accumulates severely in an iterative algorithm requiring many convergence rounds.

E. Degree Analysis — Hub and Network Structure: Theory. The degree of a vertex is the number of edges incident to it. Social networks characteristically exhibit a power-law (scale-

free) degree distribution: most vertices have few connections, while a small number of hubs have very high degree. On Facebook, node 107 has the maximum degree of 2,090, meaning it is connected to over half the entire network.

GraphFrames. Computes in-degree, out-degree, and total degree via SQL-style GROUP BY aggregation over the edges DataFrame.

GraphX. Computes degree via `graph.degrees`—an RDD-native `reduceByKey` over edge triplets, avoiding any join operations.

Results. Both frameworks matched on all degree metrics: max degree (2,090 / 6,766), average degree (87.38 / 66.04), and minimum degree. Runtime: GF 2.22 s vs GX 0.24 s — a **9.4× GraphX speedup**. GraphX's RDD reduce avoids the group-by shuffle inherent in the DataFrame approach.

F. Shortest Paths — Reachability and Small-World Structure: Theory. The shortest path between two vertices is the minimum-hop route. For social networks, average path length validates the small-world hypothesis (six degrees of separation). Both frameworks implement landmark-based BFS shortest paths, computing exact distances only to a specified set of landmark vertices.

GraphFrames. Landmarks: nodes 107, 1684, 1912. BFS is expressed as an iterative join-filter pipeline over the edges DataFrame.

GraphX. Landmarks: nodes 3437, 107. BFS is implemented via Pregel message passing, where each vertex maintains a distance map to each landmark and propagates relaxed distances to neighbors.

Results. GraphX estimated an average path length of 2.68 hops on Facebook—far below the classic 6-degrees threshold, confirming tight small-world structure. Estimated graph diameter: 5 hops. Both frameworks agree on reachability structure. Runtime: GF 2.87 s vs GX 0.76 s — a **3.80× GraphX speedup**, attributable to GraphX's efficient Pregel-based BFS over the property graph versus GraphFrames' iterative join-filter pipeline.

VI. Experimental Results

A. Metric Correctness Comparison: Six of 7 comparable metrics matched exactly (or within rounding tolerance) on both datasets. Label Propagation is the only algorithm that consistently diverges, which is a known and analytically expected property of the heuristic algorithm—not a framework defect.

VII. Performance Analysis

A. Per-Algorithm Runtime: Table VI presents per-algorithm timing for the Facebook dataset.

Fig. 2 visualizes per-algorithm runtime and Fig. 3 shows the speedup profile.

B. Scale Analysis — Performance Gap Narrows at Scale: Fig. 4 reveals a crucial finding: GraphX's advantage is strongest on medium-scale graphs (Facebook: 2.45×) but narrows significantly on large-scale graphs (Twitter: 1.22×). GraphFrames' fixed overhead from DataFrame planning becomes

TABLE IV. Metric Comparison – Facebook Combined

Metric	GraphFrames	GraphX	Status
Nodes	4,039	4,039	✓Match
Edges (undirected)	88,234	88,234	✓Match
Total Triangles	1,612,010	1,612,010	✓Match
Communities (LPA)	89	70	×Differ*
Connected Components	1	1	✓Match
Max Degree	2,090	2,090	✓Match
Average Degree	87.382	87.38	✓Match
Graph Density	N/A	0.0108	GX only
Global Clust. Coeff.	N/A	0.1292	GX only
Avg Local Clust. Coeff.	N/A	0.1437	GX only

* Divergence expected for heuristic LPA algorithm.

TABLE V. Metric Comparison – Twitter Combined

Metric	GraphFrames	GraphX	Status
Nodes	81,306	81,306	✓Match
Edges	1,342,296	1,342,296	✓Match
Total Triangles	13,082,506	13,082,506	✓Match
Communities (LPA)	1,284	1,402	×Differ*
Connected Components	1	1	✓Match
Max Degree	6,766	6,766	✓Match
Average Degree	66.0368	66.04	✓Match
Graph Density	N/A	0.0004	GX only
Global Clust. Coeff.	N/A	0.0425	GX only
Avg Local Clust. Coeff.	N/A	0.1287	GX only

* Divergence expected for heuristic LPA algorithm.

TABLE VI. Per-Algorithm Runtime – Facebook Combined

Algorithm	GF (s)	GX (s)	GX Speedup
Graph Loading	7.33	4.48	1.63×
PageRank	8.45	10.01	0.84× (GF faster)
Label Propagation	12.88	10.00	1.29×
Triangle Count	4.90	1.83	2.67×
Connected Components	11.01	0.66	16.6×
Degree Analysis	2.22	0.24	9.4×
Shortest Paths	2.87	0.76	3.80×
TOTAL	**67.35**	**28.29**	**2.38×**

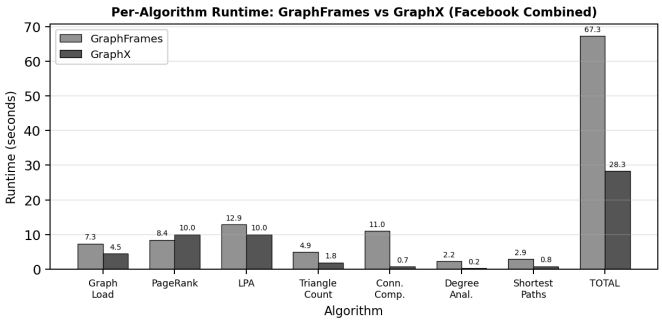

Fig. 2. Per-Algorithm Runtime: GraphFrames vs GraphX – Facebook Combined.

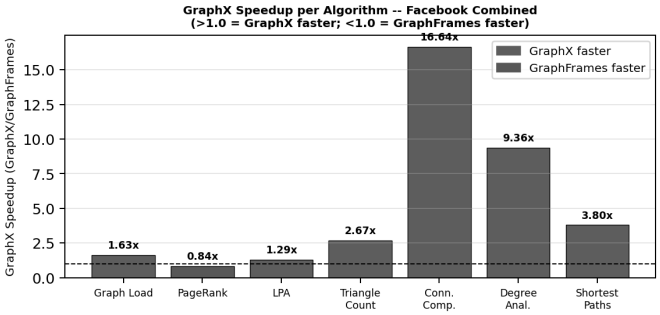

Fig. 3. GraphX Speedup per Algorithm (values >1.0 = GraphX faster; <1.0 = GraphFrames faster). Facebook Combined.

proportionally less significant as the workload grows. At very large graph sizes, Catalyst SQL optimization may close or reverse the performance gap on SQL-friendly workloads.

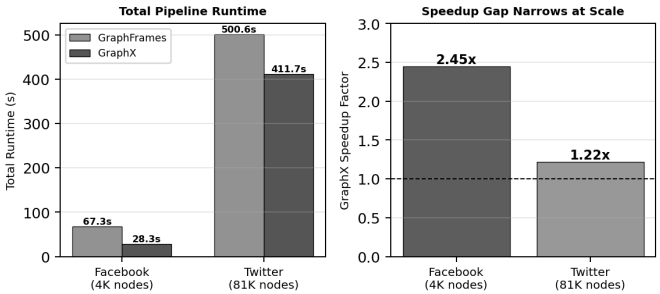

Fig. 4. Total Runtime by Dataset (left) and GraphX Speedup at Scale (right). Gap narrows from 2.45× to 1.22× at 20× larger graph.

TABLE VII. Total Pipeline Runtime and Speedup by Dataset

Dataset	GF Total (s)	GX Total (s)	Speedup
Facebook (4K nodes)	67.35	28.29	2.38×
Twitter (81K nodes)	500.62	411.71	1.22×

C. Notable Per-Algorithm Findings: Connected Components — 16.6× Speedup: The most dramatic gap. GraphX's native label-propagation completes in 0.66 s; GraphFrames requires 11.01 s due to DataFrame join overhead accumulating across every convergence iteration.

Degree Analysis — 9.4× Speedup: GraphX computes degree via an RDD-native reduce; GraphFrames uses a SQL-style GROUP BY aggregation over the edges DataFrame, incurring an additional shuffle.

PageRank — GraphFrames 16% Faster (0.84×): The only algorithm where GraphFrames outperformed GraphX. Catalyst's optimizer planned the iterative joins more efficiently for this workload, demonstrating that the performance relationship is not universally in GraphX's favour.

Triangle Count — 2.67× Speedup: GraphX's sorted adjacency intersection is more computationally efficient than GraphFrames' three-way DataFrame self-join.

Shortest Paths — 3.8× Speedup: GraphX processes BFS-style traversal more efficiently using its native Pregel property graph model compared to GraphFrames' iterative join-filter pipeline.

Label Propagation — 1.29× Speedup (with divergent output): GraphX is modestly faster; however, the output community counts differ—a known consequence of heuristic non-determinism under different execution schedules. Both outputs are valid.

D. LPA Divergence Analysis: Fig. 5 illustrates the Label Propagation community count divergence across both datasets. The divergence is attributable to three compounding sources of non-determinism in parallel LPA execution: (1) partitioned update ordering differs between RDD and DataFrame execution models; (2) tie-breaking behavior when two labels are equally common differs across implementations; and (3) the number of Pregel supersteps vs DataFrame iterations before convergence differs, affecting which local minimum the heuristic reaches. Neither community partition is incorrect.

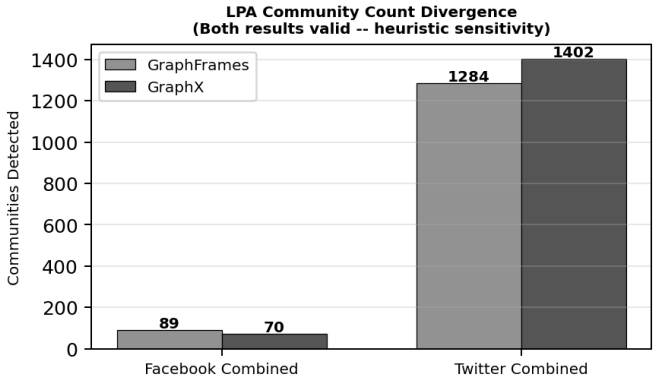

Fig. 5. LPA Community Count Divergence. Both results are valid partitions satisfying the LPA convergence criterion.

VIII. Framework Comparison — Multi-Domain Analysis

A. Graph Theory Perspective: GraphX is more *graph-native*: it closely mirrors classical graph processing systems through vertex/edge RDD structures, neighborhood aggregation, and Pregel-style BSP computation. GraphFrames is more *graph-relational*: it treats graph structure as a pair of joinable relations, enabling composition with external metadata tables, activity logs, and reporting layers—exactly the architecture required in production analytics pipelines.

B. Data Structures and Algorithms Perspective: GraphX aligns with graph-native DSA: property graphs, iterative algorithms, message passing, Pregel execution. GraphFrames aligns with relational DSA applied to graph-shaped data: DataFrame vertices/edges, SQL joins, group-by aggregation, Catalyst-optimized planning. This explains why GraphX is more natural for custom graph algorithms, while GraphFrames is more natural for analytics pipelines.

C. Distributed Systems Perspective: GraphX sits on the graph-engine side of Spark (RDD transformations, in-memory caching, graph-native primitives). GraphFrames sits on the analytics-workflow side (SQL planning, columnar DataFrame execution, Catalyst optimization). GraphX wins on iterative graph-native runtime; GraphFrames wins on workflow flexibility, ecosystem integration, and analytics usability.

D. Summary Comparison Table:

TABLE VIII. Multi-Dimensional Framework Comparison

Dimension	GraphFrames	GraphX
Core Model	DataFrame + SQL	RDD + Property Graph
Language	Python, Scala, Java	Scala (primary)
Motif Search	Native DSL	Manual logic required
Optimization	Catalyst planner	Graph-native RDD ops
Perf. (small)	Slower	Faster
Perf. (large)	Competitive	Moderate advantage
PageRank	Faster (16%)	Slower
Conn. Comp.	16.6× slower	Native advantage
Future	Active (v0.10.0)	Deprecated (Spark 4.x)

IX. Case Study: KSRTC Smart Transit System

A. Overview: The KSRTC Smart Transit System [**?**] demonstrates a practical deployment of Spark-based graph analytics for large-scale public transportation. Urban transportation systems generate massive volumes of heterogeneous data including route structures, GPS traces, and traffic conditions. Traditional systems fail to process such data efficiently due to scalability limitations. This work presents a distributed analytics framework for KSRTC transportation data using Apache Spark, GraphX, and Kafka, enabling scalable data processing, graph-based modeling, and real-time analytics.

B. System Architecture and Pipeline: The system is designed as a multi-layered architecture consisting of data ingestion, storage, processing, graph analytics, and visualization layers. The five-stage pipeline is shown in Fig. 6.

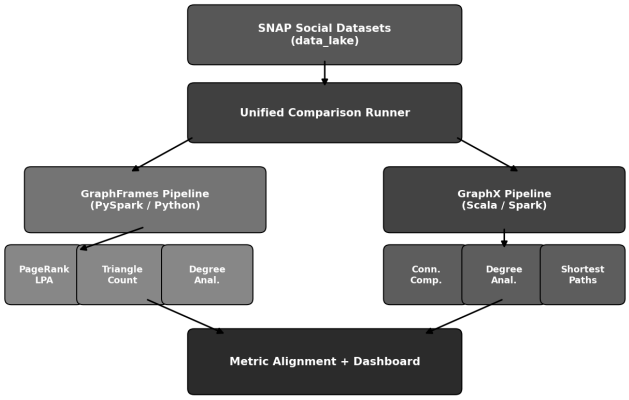

Fig. 6. KSRTC Data Processing Pipeline: Raw GPS Data → Spark ETL → GraphX → MongoDB → Dashboard.

The data ingestion layer collects raw transportation datasets including route information and simulated GPS streams, stored in a distributed file system for fault tolerance and scalability. Apache Spark performs ETL operations; processed data is then transformed into a graph representation using GraphX, enabling efficient execution of graph algorithms. Final results are exposed through APIs and visualized on an interactive dashboard.

C. Data Processing: Raw datasets contain inconsistencies such as missing values and duplicate entries, addressed using distributed Spark transformations. Feature engineering derives meaningful attributes including speed, route frequency, and temporal patterns.

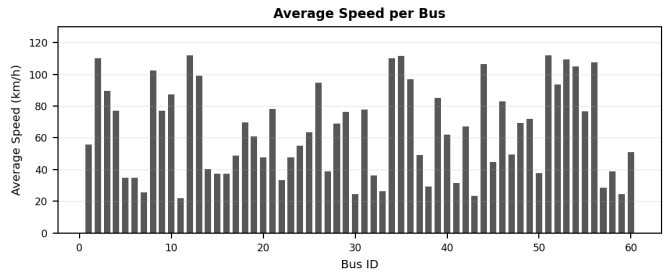

Fig. 7. Average Speed Distribution Across Buses.

Fig. 7 shows the per-bus average speed distribution across the fleet. Most buses operate between 40–100 km/h, with variation attributable to route type (express vs. city service) and time-of-day congestion patterns. The following PySpark snippet illustrates the core ETL transformation applied to the KSRTC CSV dataset:

```
df = spark.read.csv("ksrtc.csv", header=True)
df = df.dropna()
df = df.withColumn("speed",
    col("distance") / col("time"))
df.write.parquet("processed_data")
```

D. Graph Analytics on the Route Network: The KSRTC transportation network is modeled as a property graph: bus stops are vertices, direct route connections are directed edges. Fig. 8 shows a sampled subgraph of this network.

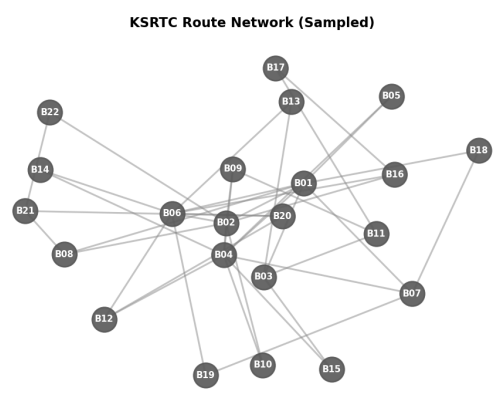

Fig. 8. Graph Representation of KSRTC Route Network (Sampled). Vertices = bus stops; edges = direct routes.

Three graph algorithms are applied. **PageRank** identifies crit-

ical transit hubs with highest connectivity centrality. **Connected Components** verifies network integrity and detects isolated sub-networks. **Shortest Paths** enables route-level distance optimization between stops.

1) Shortest Path Analysis: Shortest path computation is applied from a fixed landmark stop to all reachable stops in the network. Fig. 9 shows the resulting distance distribution.

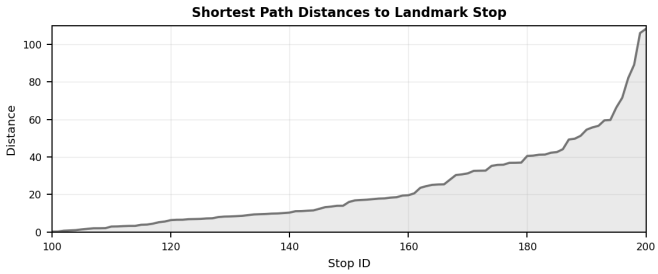

Fig. 9. Shortest Path Distances to Landmark Stop. The monotonic growth pattern confirms a well-connected network with few isolated outliers.

2) PageRank — Transit Hub Identification: PageRank scores are computed using GraphX's native Pregel-based implementation. The result identifies transit hubs—stops with disproportionately high centrality that serve as critical interchange nodes. Fig. 10 shows the PageRank distribution across sampled stops.

```
val ranks = graph.pageRank(0.0001)
val topStops = ranks.vertices
    .sortBy(_._2, false)
```

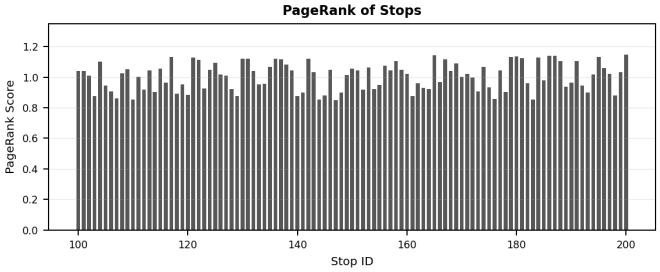

Fig. 10. Top Bus Stops Identified by PageRank. Stops with scores above 1.0 are candidate transit hubs.

E. Results and Discussion:

1) Speed Distribution: Fig. 11 shows the global speed distribution across all routes. A multimodal pattern emerges: peaks around 35 km/h (city-center routes with heavy stops), 55 km/h (suburban corridors), and 90–110 km/h (express highways). This distribution directly informs congestion zone identification and schedule optimization.

2) Traffic Density Heatmap: Fig. 12 shows the spatial traffic density heatmap over the KSRTC service region. High-density zones (dark red) indicate congestion hotspots—predominantly in the 13–16°N latitude and 76–78°E longitude range, corresponding to urban agglomeration zones. This heatmap directly informs route priority and dynamic schedule adjustment.

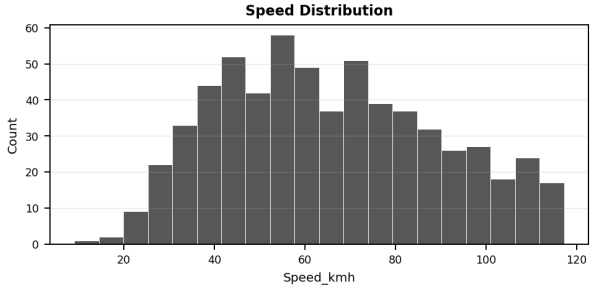

Fig. 11. Distribution of Bus Speeds Across All Routes.

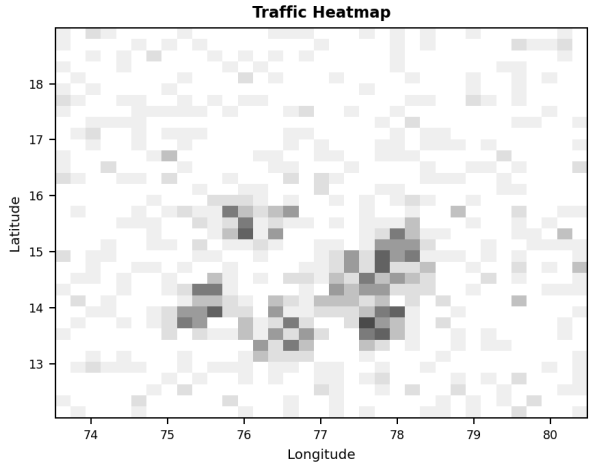

Fig. 12. Traffic Density Heatmap over KSRTC Service Region.

F. Real-Time Processing with Kafka: The system integrates Apache Kafka to simulate real-time GPS data streams. These streams are processed using Spark Streaming to provide low-latency analytics and continuous monitoring of route performance and traffic conditions.

```
producer.send("gps_topic", data)
stream = spark.readStream \
    .format("kafka") \
    .option("subscribe", "gps_topic") \
    .load()
```

The Kafka → Spark Streaming → GraphX pipeline enables sub-second update latency for traffic state, making it suitable for real-time operations center dashboards.

G. Framework Selection Rationale for KSRTC: GraphX is the correct engine for KSRTC's core computation because: (1) route optimization requires iterative Dijkstra-style shortest-path computation, native to Pregel; (2) PageRank for transit hub identification runs efficiently on the RDD property graph model without DataFrame join overhead; (3) Kafka + Spark Streaming + GraphX RDD pipelines integrate naturally for real-time GPS analytics; and (4) transportation workloads are predominantly graph-native with minimal relational join requirements.

While GraphX is correct for core computation, GraphFrames would benefit an extended smart-city analytics layer: SQL joins between route-graph metrics and passenger metadata/fare tables; motif search for detecting route patterns (loops, bottleneck junc-

tions); Python-accessible analytics for operations teams; and dashboard queries combining graph metrics with operational data.

KSRTC Conclusion: GraphX is the right choice for route computation. GraphFrames is the right complement for a broader analytics layer combining graph structure with tabular operational data and dashboard workflows.

X. Strengths and Weaknesses

TABLE IX. Framework Strengths and Weaknesses

	GraphFrames	**GraphX**
Strengths	Natural DataFrame/SQL integration; Python-friendly; motif finding built-in; modern analytics fit; broader algorithm surface	Graph-native execution model; fastest on iterative algorithms; custom graph-algorithm control; graph-theory alignment
Weaknesses	Slower on graph-native workloads; high join overhead; non-deterministic LPA	Scala-centric; less flexible for relational analytics; flagged for deprecation

XI. Threats to Validity

Dataset scope: Only two datasets were used. Results may not generalize to heterogeneous, bipartite, or directed graphs with different structural properties.

Environment scope: Experiments reflect a single-machine Spark session. Cluster-scale behavior across multiple executors may alter the performance gap due to different shuffle and communication cost profiles.

Algorithm scope: The comparison covers 6 algorithms. Both frameworks expose broader feature surfaces not directly measured here (e.g., betweenness centrality, k-core decomposition, SVD).

LPA variability: Because LPA is heuristic, community count differences are analytically meaningful but not evidence of a structural bug. Multiple valid partitions can exist for the same graph.

Asymmetric metrics: Graph density and clustering coefficients are computed only by GraphX in this pipeline, limiting symmetric comparison on those metrics.

XII. Future Work

- Include more SNAP datasets (bipartite, heterogeneous, directed graphs).
- Benchmark GraphFrames motif-finding vs custom GraphX triplet logic directly.
- Add memory consumption and garbage-collection profiling.
- Evaluate cluster-scale behavior on a multi-node Spark deployment.
- Compare both frameworks with external systems (GraphBLAS, Neo4j, Giraph).
- Implement betweenness centrality, k-core decomposition, and SVD++ in both frameworks.
- Integrate real KSRTC GPS streams with the Spark Streaming + GraphX pipeline.

XIII. Conclusion

This paper compared GraphFrames and GraphX for social network analysis on Apache Spark using real measured evidence from an implemented pipeline on two SNAP datasets spanning two orders of magnitude in graph scale. The findings are as follows.

Correctness: 6 of 7 comparable metrics matched exactly on both datasets. Label Propagation divergence is expected and analytically explained.

Performance – Facebook: GraphX: 28.29 s vs GraphFrames: 67.35 s (**2.38× faster**).

Performance – Twitter: GraphX: 411.71 s vs GraphFrames: 500.62 s (**1.22× – gap narrows at scale**).

Graph-Native Algorithms: GraphX dominates: Connected Components (16.6×), Degree Analysis (9.4×), Shortest Paths (3.8×), Triangle Count (2.67×).

SQL-Friendly Workloads: GraphFrames outperforms GraphX on PageRank (0.84×), indicating Catalyst's optimizer can be superior for join-heavy iterative computation.

Final conclusion: GraphX is the stronger graph-computation engine for iterative graph workloads. GraphFrames is the stronger graph-analytics workflow framework for modern Spark-based social network projects. The two frameworks are complementary—not substitutes. Practitioners should consider GraphX for custom graph algorithms and high-throughput iterative computation, and GraphFrames for analytics pipelines requiring DataFrame integration, motif search, Python accessibility, and long-term ecosystem compatibility.

Acknowledgment

The authors thank the Stanford Network Analysis Project (SNAP) for providing publicly accessible social network datasets used in this study.

References

[1] J. E. Gonzalez, R. S. Xin, A. Dave, D. Crankshaw, M. J. Franklin, and I. Stoica, "GraphX: Graph processing in a distributed dataflow framework," in *Proc. USENIX OSDI*, 2014.

[2] A. Dave, A. Deshpande, M. J. Franklin, J. M. Hellerstein, I. Stoica, and A. Ghodsi, "GraphFrames: An integrated API for mixing graph and relational queries," in *Proc. GRADES*, 2016.

[3] G. Malewicz et al., "Pregel: A system for large-scale graph processing," in *Proc. ACM SIGMOD*, 2010.

[4] M. Armbrust et al., "Spark SQL: Relational data processing in Spark," in *Proc. ACM SIGMOD*, 2015.

[5] GraphFrames Project, "GraphFrames is back," Aug. 2025. [Online]. Available: https://graphframes.io/05-blog/1000-graphframes-is-back.html

[6] Apache Spark, "SPARK-50857: Mark GraphX as deprecated," Apache JIRA, 2025. [Online]. Available: https://issues.apache.org/jira/browse/SPARK-50857

[7] GraphFrames Project, "Quick Start," 2026. [Online]. Available: https://graphframes.io/02-quick-start/02-quick-start.html

[8] GraphFrames Project, "Benchmarks," 2026. [Online]. Available: https://graphframes.io/01-about/03-benchmarks.html

[9] X. Yang et al., "Revisiting graph analytics benchmark," in *Proc. ACM SIGMOD*, 2025. doi:10.1145/3725345.

[10] E.-S. Apostol, A.-C. Cojocaru, and C.-O. Truica, "Large-scale graphs community detection using Spark GraphFrames," arXiv:2408.03966, 2024.

[11] J. Leskovec and A. Krevl, “SNAP Datasets: Stanford large network dataset collection,” Jun. 2014. [Online]. Available: https://snap.stanford.edu/data/
[12] GraphFrames Project, “Motif Finding,” 2026. [Online]. Available: https://graphframes.io/04-user-guide/04-motif-finding.html